Practical Work Number 1

Specification(C++)

Type Aliases

```
typedef unsigned int Vertex;
typedef unsigned int EdgeCost;

typedef std::pair<Vertex, Vertex> Edge;

typedef std::vector<Vertex> Vertices;

typedef std::map<Vertex, Vertices> Outbound;
typedef std::map<Vertex, Vertices> Inbound;
typedef std::map<Edge, EdgeCost> EdgeCosts;

typedef Outbound::const_iterator OutboundIterator;
typedef EdgeCosts::const_iterator EdgesIterator;
typedef Vertices::const_iterator VerticesIterator;
```

`ADTDirectedGraph`

A class named **ADTDirectedGraph** representing an abstract data type directed graph.

The class **ADTDirectedGraph** has the following **methods**:

```
explicit ADTDirectedGraph
```

```
Construct a new ADTDirectedGraph object (directed graph with no capacity).
```

```
unsigned int nbVertices() const
```

```
Returns the number of vertices in the graph.
```

```
unsigned int nbEdges() const
```

Returns the number of edges in the graph.

```
bool isEdge(Vertex first, Vertex second) const
```

Checks if an edge between 2 vertexes exist.

```
unsigned int getInDegree(Vertex vertex) const
```

Get the in-degree of a vertex.

```
unsigned int getOutDegree(Vertex vertex) const
```

Get the out-degree of a vertex.

```
bool addEdge(Edge edge, EdgeCost cost)
```

Adds an edge to the graph.

```
bool removeEdge(Edge edge)
```

Removes an edge from the graph.

```
bool addVertex(Vertex vertex)
```

Adds a vertex to the graph.

```
bool removeVertex(Vertex vertex)
```

Removes a vertex from the graph.

```
ADTDirectedGraphIterator parseVertices() const
```

Returns the iterator for the graph's vertices.

```
VerticesIterator parseOutboundOfGivenVertexBegin(Vertex vertex) const
```

Returns the iterator for the outbound neighbors of a given vertex from the begin

```
VerticesIterator parseOutboundOfGivenVertexEnd(Vertex vertex) const
```

Returns the iterator for the outbound neighbors of a given vertex from the end.

```
VerticesIterator parseInboundOfGivenVertexEnd(Vertex vertex) const
```

Returns the iterator for the inbound neighbors of a given vertex from the end.

```
VerticesIterator parseInboundOfGivenVertexBegin(Vertex vertex) const
```

Returns the iterator for the inbound neighbors of a given vertex from the beginn

```
EdgesIterator parseEdgesBegin() const
```

Returns the iterator for the edges (with costs) from the beginning of the iterat

```
EdgesIterator parseEdgesEnd() const
```

Returns the iterator for the edges (with costs) from the end of the iteratable.

ADTDirectedGraphIterator

A class named **ADTDirectedGraphIterator** representing an auxiliary class for ADTDirectedGraph iteration.

The class **ADTDirectedGraphIterator** has the following **methods**:

```
void first()

Sets the iterator to the first element in the iteratable.
```

```
void next()

Sets the iterator to the next element of the iteratable.
```

```
Vertex getCurrent();

Gets the current Vertex of the iteratable.
```

```
bool valid() const;

Checks if the current element of the iterator is valid.
```

MenuUI

A class named **MenuUI** representing the main user interface. Very importantly, this contains the **start** method.

The class **MenuUI** has the following **methods**:

```
explicit MenuUI(ADTDirectedGraph &g)
```

Constructs a new MenuUI object using a ADTDirectedGraph reference.

```
void start();
```

Starts the menu application :).

```
void displayMenuOptions() const;
```

Displays the available menu options.

```
unsigned int getUserUnsignedInt(const std::string &label) const;
```

Gets an unsigned int from the console, given a descriptive short label.

```
Vertex getUserVertex() const
```

Gets a Vertex from the console.

```
Edge getUserEdge() const;

Gets an Edge from the console.
```

```
EdgeCost getUserEdgeCost() const;

Get an Edge Cost from the console.
```

```
std::string getUserFilename() const;

Gets a filename from the console. In retrospect, this should be relabeled as getl
```

```
MenuController controller() const;

Returns a dynamically instantiated auxiliary class MenuController that holds all
```

MenuController

A class named **MenuController** representing the auxiliary controller class for the **MenuUI** class.

The class **MenuController** has the following **methods**:

```
void displayCurrentNbVertices() const;
```

Displays the current number of vertices in the graph.

```
void displayCurrentNbEdges() const;
```

Displays the current number of edges in the graph.

```
void displayGraphVertices() const;
```

Displays the vertices of the graph.

```
void displayGraphEdges() const;
```

Displays the edges of the graph.

```
void displayOutboundEdgesOfGivenVertex() const;
```

Displays the outbound neighbors of a given vertex.

```
void displayInboundEdgesOfGivenVertex() const;
```

Displays the inbound neighbors of a given vertex.

```
void checkIfGivenEdgeIsGraphEdge() const;
```

Displays if a given edge is in the graph or not.

```
void displayInDegreeOfGivenVertex() const;
```

Displays the indegree of a given vortex;

```
void displayOutDegreeOfGivenVertex() const;
```

Displays the outdegree of a given vortex;

```
void addGivenVertexToGraph();
```

Add a given user vertex to the graph.

```
void removeGivenVertexFromGraph();
```

Removes a given user vertex from the graph.

```
void addGivenEdgeToGraph();
```

Adds a given user edge to the graph.

```
void removeGivenEdgeFromGraph();
```

Removes a given user edge from the graph.

```
void generateRandomGraphOfGivenNbVerticesAndNbEdges();
```

Generates a random graph of given number of vertices and edges. This indeed overı

```
void readGraphDataFromGivenFile();
```

Reads a graph data from a given file by filepath. This also indeed overwritess tl

```
void writeGraphDataToGivenFile();
```

Writes to a given file by filepath the current graph. Keep in mind that this als

Helpers

A class named **Helpers** representing the auxiliary class for the **MenuController** class containing some useful helper **ADTDirectedGraph** methods.

The class **Helpers** has the following **methods**:

```
explicit Helperss()
```

```
Constructs a new Helpers object. This is optional as all of the Helper class metl
```

```
static bool readGraph(ADTDirectedGraph &graph, const std::string &filename);
```

```
Reads a graph from a specified text file. Keep in mind that this is relative to t
```

```
static bool writeGraph(const ADTDirectedGraph &graph, const std::string &filename);
```

```
Writes to a text file with a specified filename the graph. Same observations as a
```

```
static ADTDirectedGraph generateRandomGraph(unsigned int nbVertices, unsigned int nbEdges);
```

```
Generates a random graph with nbVertices number of vertices and nbEdges number o
```

Implementation (C++)

`ADTDirectedGraph`

Class **ADTDirectedGraph** has the following data members:

```
Outbound outbound; -> map of outbound neighbors of key vertices
Inbound inbound; -> map of inbound neighbors of key vertices
EdgeCosts costs; -> map of edge costs of key edges
```

Class **ADTDirectedGraph** is friends with **ADTDirectedGraphIterator** class, with the observation of forwardly declarating it so no dependency cycles occur.

ADTDirectedGraph class **methods**:

```cpp
ADTDirectedGraph::ADTDirectedGraph()
{
    // handle outbound
    this->outbound = Outbound{};

    // handle inbound
    this->inbound = Inbound{};

    // handle costs
    this->costs = EdgeCosts{};
}
```

```cpp
unsigned int ADTDirectedGraph::nbVertices() const
{
    unsigned int nbVertices = 0;

    for (const auto &_ : this->outbound)
    {
        nbVertices++;
    }

    return nbVertices;
}
```



```cpp
unsigned int ADTDirectedGraph::nbEdges() const
{
    return this->costs.size();
}
```

```
bool ADTDirectedGraph::isEdge(Vertex first, Vertex second) const
{

    if (this->outbound.count(first) == 0 || this->inbound.count(second) == 0)
    {
        return false;
    }

    Vertices outboundNeigh = this->outbound.at(first);

    bool found = false;

    for (const Vertex &vertex : outboundNeigh)
    {
        if (vertex == second)
        {
            found = true;
            break;
        }
    }

    return found;
}
```

```
unsigned int ADTDirectedGraph::getInDegree(Vertex vertex) const
{
    return this->outbound.at(vertex).size();
}
```

```
unsigned int ADTDirectedGraph::getOutDegree(Vertex vertex) const
{
    return this->inbound.at(vertex).size();
}
```

```
bool ADTDirectedGraph::addEdge(Edge edge, EdgeCost cost)
{

    // check if edge vertices exist in the graph
    if (this->outbound.count(edge.first) == 0 || this->outbound.count(edge.second) == 0)
    {
        return false;
    }

    // check if edge is already in the graph
    // we can more easily do this by checking the costs map
    if (this->costs.count(edge) == 1)
    {
        return false;
    }

    // handle outbound
    this->outbound.at(edge.first).push_back(edge.second);
    // handle inbound
    this->inbound.at(edge.second).push_back(edge.first);
    // handle the cost map
    this->costs[edge] = cost;

    return true;
}
```

```
bool ADTDirectedGraph::removeEdge(Edge edge)
{

    // check if edge exists in the graph
    // easiest way to do this is through the costs
    if (this->costs.count(edge) == 0)
    {
        return false;
    }

    // handle outbound
    unsigned int outIndex = -1;
    Vertices &outboundNeigh = this->outbound.at(edge.first);
    for (unsigned int i = 0; i < outboundNeigh.size(); i++)
    {
        if (outboundNeigh.at(i) == edge.second)
        {
            outIndex = i;
            break;
        }
    }
    // remove the vertex from the outbound neighbors of first
    outboundNeigh.erase(outboundNeigh.begin() + outIndex);

    // handle inbound
    // basically the same thing as above but we change the source and dest
    unsigned int inIndex = -1;
    Vertices &inboundNeigh = this->inbound.at(edge.second);
    for (unsigned int i = 0; i < inboundNeigh.size(); i++)
    {
        if (inboundNeigh.at(i) == edge.first)
        {
            inIndex = i;
            break;
        }
    }
    // remove the vertex from the inbound neighbors of second
    inboundNeigh.erase(inboundNeigh.begin() + inIndex);

    // handle removing the edge from the costs
    this->costs.erase(edge);

    return true;
}
```

```
bool ADTDirectedGraph::addVertex(Vertex vertex)
{
    // check if vertex is in graph already
    if (this->outbound.count(vertex) != 0)
    {
        return false;
    }

    // handle outbound
    this->outbound[vertex] = Vertices{};

    // handle inbound
    this->inbound[vertex] = Vertices{};

    return true;
}
```

```
bool ADTDirectedGraph::removeVertex(Vertex vertex)
{

    // check if vertex is in the graphs
    if (this->outbound.count(vertex) == 0)
    {
        return false;
    }

    // handle inbound first
    Vertices &outboundNeigh = this->outbound.at(vertex);

    // we check in the inbound of the current vertex if we have any matches then delete said ma
    for (const Vertex &currOutVertex : outboundNeigh)
    {
        Vertices &currInboundNeigh = this->inbound.at(currOutVertex);

        for (unsigned int i = 0; i < currInboundNeigh.size(); i++)
        {
            if (currInboundNeigh.at(i) == vertex)
            {
                currInboundNeigh.erase(currInboundNeigh.begin() + i);
                break;
            }
        }
    }

    // now we delete the key from the outbound itself
    this->outbound.erase(vertex);

    // handle inbound
    Vertices &inboundNeigh = this->inbound.at(vertex);

    // we check in the outbound of the current vertex if we have any matches then delete said r
    for (const Vertex &currInVertex : inboundNeigh)
    {
        Vertices &currOutboundNeigh = this->outbound.at(currInVertex);

        for (unsigned int i = 0; i < currOutboundNeigh.size(); i++)
        {
            if (currOutboundNeigh.at(i) == currInVertex)
            {
                currOutboundNeigh.erase(currOutboundNeigh.begin() + i);
                break;
            }
        }
```

```
    }

    // now we delete the key from the inbound itself
    this->inbound.erase(vertex);

    // handle costs

    // we iterate over the costs then if the key (type Edge) includes match we delete the entry
    for (const auto &[edge, cost] : this->costs)
    {
        if (edge.first == vertex || edge.second == vertex)
        {
            this->costs.erase(edge);
        }
    }

    return true;
}
```

```
ADTDirectedGraphIterator ADTDirectedGraph::parseVertices() const
{

    return ADTDirectedGraphIterator(*this);
}
```

```
VerticesIterator ADTDirectedGraph::parseOutboundOfGivenVertexBegin(Vertex vertex) const
{

    if (this->outbound.count(vertex) == 0)
    {
        throw std::exception();
    }

    return this->outbound.at(vertex).begin();
}
```

```
VerticesIterator ADTDirectedGraph::parseOutboundOfGivenVertexEnd(Vertex vertex) const
{

    if (this->outbound.count(vertex) == 0)
    {
        throw std::exception();
    }

    return this->outbound.at(vertex).end();
}
```

```
VerticesIterator ADTDirectedGraph::parseInboundOfGivenVertexEnd(Vertex vertex) const
{

    if (this->inbound.count(vertex) == 0)
    {
        throw std::exception();
    }

    return this->inbound.at(vertex).end();
}
```

```
VerticesIterator ADTDirectedGraph::parseInboundOfGivenVertexBegin(Vertex vertex) const
{

    if (this->inbound.count(vertex) == 0)
    {
        throw std::exception();
    }

    return this->inbound.at(vertex).begin();
}
```

```
EdgesIterator ADTDirectedGraph::parseEdgesBegin() const
{
    return this->costs.begin();
}
```

```
EdgesIterator ADTDirectedGraph::parseEdgesEnd() const
{
    return this->costs.end();
}
```

ADTDirectedGraphIterator

Class **ADTDirectedGraphIterator** has the following data members:

```
    const ADTDirectedGraph &graph; -> the graph to be iterated
    OutboundIterator currentVerticesIterator; -> current iterator
```

Class **ADTDirectedGraphIterator** is friends with **ADTDirectedGraph** class since it is accessing it's outbound data member.

ADTDirectedGraph class **methods**:

```
void ADTDirectedGraphIterator::first()
{
    if (this->graph.nbVertices() == 0)
    {
        throw std::exception();
    }
    this->currentVerticesIterator = this->graph.outbound.begin();
}
```

```
void ADTDirectedGraphIterator::next()
{
    if (!this->valid())
    {
        throw std::exception();
    }

    this->currentVerticesIterator = std::next(this->currentVerticesIterator);
    return;
}
```

```
Vertex ADTDirectedGraphIterator::getCurrent()
{
    if (!this->valid())
    {
        throw std::exception();
    }

    return this->currentVerticesIterator->first;
}
```

```
bool ADTDirectedGraphIterator::valid() const
{
    return this->currentVerticesIterator != this->graph.outbound.end();
}
```

MenuUI

Class **MenuUI** has the following data members:

```
ADTDirectedGraph &graph; -> mutable reference to a graph

unsigned short userMenuOption; -> the current user option
```

Class **MenuUI** is friends with **MenuController** class, with the observation of forwardly declarating it so no dependency cycles occur.

MenuUI class **methods**:

```
MenuUI::MenuUI(ADTDirectedGraph &g) : graph(g)
{
    this->userMenuOption = 0;
}
```

```
void MenuUI::displayMenuOptions() const
{
    std::cout << "\n---------MENU OPTIONS---------\n";
    std::cout << "\n---------GRAPH PROPERTIES---------\n";
    std::cout << "1: Get the current number of Vertices of the Graph.\n";
    std::cout << "2: Get the current number of Edges of the Graph.\n";
    std::cout << "3: Display the Vertices of the Graph.\n";
    std::cout << "4: Display the Edges of the Graph (with cost included).\n";
    std::cout << "5: Display the Outbound Edges of a given Vertex.\n";
    std::cout << "6: Display the Inbound Edges of a given Vertex.\n";
    std::cout << "7: Check if a given Edge is an Edge of the Graph.\n";
    std::cout << "8: Get the in-degree of a given Vertex.\n";
    std::cout << "9: Get the out-degree of a given Vertex.\n";
    std::cout << "\n---------GRAPH MUTATION---------\n";
    std::cout << "10: Add a given Vertex to the Graph.\n";
    std::cout << "11: Remove a given Vertex from the Graph.\n";
    std::cout << "12: Add a given Edge to the Graph.\n";
    std::cout << "13: Remove a given Edge from the Graph.\n";
    std::cout << "\n---------UTILITIES---------\n";
    std::cout << "14: Generate a random Graph with given nb of vertices and given nb of edges.
    std::cout << "15: Read from a given file graph data. This updates the current Graph.\n";
    std::cout << "16: Write to a given filename graph data.\n";
    std::cout << "17: Exit program.\n";
    std::cout << std::endl;
}
```

```
unsigned int MenuUI::getUserUnsignedInt(const std::string &label) const
{
    unsigned int value;
    long long catcher;

    while (true)
    {

        std::cout << label;
        if (std::cin >> catcher && catcher >= 0)
        {
            value = catcher;
            std::cout << std::endl;
            return value;
        }
        std::cin.clear();
        std::cout << "Invalid input, expected an unsigned int." << std::endl;
    }
}




Vertex MenuUI::getUserVertex() const
{
    Vertex vertex;
    long long vertexCatcher;
    while (true)
    {
        std::cout << "Vertex (unsigned int): ";

        if (std::cin >> vertexCatcher && vertexCatcher >= 0)
        {
            vertex = vertexCatcher;
            std::cout << std::endl;
            return vertex;
        }
        std::cin.clear();
        std::cout << "Invalid input, expected a Vertex (unsigned int)." << std::endl;
    }
}
```

```
Edge MenuUI::getUserEdge() const
{
    Edge edge;
    long long firstCatcher;
    long long secondCatcher;
    while (true)
    {
        std::cout << "Edge (unsigned int, unsigned int): ";

        if (std::cin >> firstCatcher >> secondCatcher && firstCatcher >= 0 && secondCatcher >=
        {
            edge = {firstCatcher, secondCatcher};
            std::cout << std::endl;
            return edge;
        }
        std::cin.clear();
        std::cout << "Invalid input, expected an Edge (ex: 2 5)." << std::endl;
    }

    return edge;
}
```

```
EdgeCost MenuUI::getUserEdgeCost() const
{
    EdgeCost cost;
    long long costCatcher;
    while (true)
    {
        std::cout << "Cost (unsigned int): ";

        if (std::cin >> costCatcher && costCatcher >= 0)
        {
            cost = costCatcher;
            std::cout << std::endl;
            return cost;
        }
        std::cin.clear();
        std::cout << "Invalid input, expected an EdgeCost (unsigned int)." << std::endl;
    }
}


std::string MenuUI::getUserFilename() const
{
    std::string filename;

    while (true)
    {
        std::cout << "Filename: ";

        if (std::cin >> filename)
        {
            std::cout << std::endl;
            return filename;
        }
        std::cin.clear();
        std::cout << "Invalid input, expected a Filename (string)." << std::endl;
    }
}
```

```
void MenuUI::start()
{

    MenuController menuController = this->controller();

    while (true)
    {
        this->displayMenuOptions();
        this->userMenuOption = getUserUnsignedInt("Menu Option: ");
        switch (userMenuOption)
        {
        case 1:
            menuController.displayCurrentNbVertices();
            break;
        case 2:
            menuController.displayCurrentNbEdges();
            break;
        case 3:
            menuController.displayGraphVertices();
            break;
        case 4:
            menuController.displayGraphEdges();
            break;
        case 5:
            menuController.displayOutboundEdgesOfGivenVertex();
            break;
        case 6:
            menuController.displayInboundEdgesOfGivenVertex();
            break;
        case 7:
            menuController.checkIfGivenEdgeIsGraphEdge();
            break;
        case 8:
            menuController.displayInDegreeOfGivenVertex();
            break;
        case 9:
            menuController.displayOutDegreeOfGivenVertex();
            break;
        case 10:
            menuController.addGivenVertexToGraph();
            break;
        case 11:
            menuController.removeGivenVertexFromGraph();
            break;
        case 12:
            menuController.addGivenEdgeToGraph();
```

```
            break;
        case 13:
            menuController.removeGivenEdgeFromGraph();
            break;
        case 14:
            menuController.generateRandomGraphOfGivenNbVerticesAndNbEdges();
            break;
        case 15:
            menuController.readGraphDataFromGivenFile();
            break;
        case 16:
            menuController.writeGraphDataToGivenFile();
            break;
        case 17:
            std::cout << "Exiting..." << std::endl;
            return;
        default:
            break;
        }
    }
}


MenuController MenuUI::controller() const
{
    return MenuController(*this, this->graph);
}
```

MenuController

Class **MenuController** has the following data members:

```
    const MenuUI &ui; -> reference to the ui
    ADTDirectedGraph &graph; -> reference to the graph
```

Class **MenuController** is friends with **MenuUI** class since it uses it's graph data member.

MenuController class **methods**:

```
MenuController::MenuController(const MenuUI &u, ADTDirectedGraph &g) : ui(u), graph(g) {}
```

```
void MenuController::displayCurrentNbVertices() const
{
    std::cout << "Number of Vertices: " << this->graph.nbVertices() << std::endl;
}





void MenuController::displayCurrentNbEdges() const
{
    std::cout << "Number of Edges: " << this->graph.nbEdges() << std::endl;
}





void MenuController::displayGraphVertices() const
{

    if (this->graph.nbVertices() == 0)
    {
        std::cout << "No Vertices in Graph to display." << std::endl;
        return;
    }

    std::cout << "Vertices: \n";

    ADTDirectedGraphIterator it = this->graph.parseVertices();

    it.first();
    while (it.valid())
    {
        std::cout << it.getCurrent() << std::endl;
        it.next();
    }

    std::cout << std::endl;
}
```

```
void MenuController::displayGraphEdges() const
{

    if (this->graph.nbEdges() == 0)
    {
        std::cout << "No Edges in Graph to display." << std::endl;
        return;
    }

    std::cout << "Edges: \n";

    EdgesIterator startIt = this->graph.parseEdgesBegin();
    EdgesIterator endIt = this->graph.parseEdgesEnd();

    while (startIt != endIt)
    {

        std::cout << startIt->first.first << ' ' << startIt->first.second << ' ' << startIt->se

        ++startIt;
    }

    std::cout << std::endl;
}
```

```
void MenuController::displayOutboundEdgesOfGivenVertex() const
{
    Vertex vertex = this->ui.getUserVertex();

    std::cout << "Outbound Edges: \n";

    try
    {
        VerticesIterator startIt = this->graph.parseOutboundOfGivenVertexBegin(vertex);
        VerticesIterator endIt = this->graph.parseOutboundOfGivenVertexEnd(vertex);

        while (startIt != endIt)
        {

            std::cout << *startIt << std::endl;

            ++startIt;
        }

        std::cout << std::endl;
    }
    catch (const std::exception &e)
    {
        std::cout << "Vertex doesn't exist in graph." << std::endl;
    }
}
```

```
void MenuController::displayInboundEdgesOfGivenVertex() const
{
    Vertex vertex = this->ui.getUserVertex();

    std::cout << "Inbound Edges: \n";

    try
    {

        VerticesIterator startIt = this->graph.parseInboundOfGivenVertexBegin(vertex);
        VerticesIterator endIt = this->graph.parseInboundOfGivenVertexEnd(vertex);

        while (startIt != endIt)
        {

            std::cout << *startIt << std::endl;

            ++startIt;
        }

        std::cout << std::endl;
    }
    catch (const std::exception &e)
    {
        std::cout << "Vertex doesn't exist in graph." << std::endl;
    }
}
```

```
void MenuController::checkIfGivenEdgeIsGraphEdge() const
{
    Edge edge = this->ui.getUserEdge();

    if (this->graph.isEdge(edge.first, edge.second))
    {
        std::cout << "Given Edge is IN the graph." << std::endl;
    }
    else
    {
        std::cout << "Given Edge is NOT in the graph." << std::endl;
    }

    std::cout << std::endl;
}
```

```
void MenuController::displayInDegreeOfGivenVertex() const
{
    Vertex vertex = this->ui.getUserVertex();

    try
    {
        unsigned int degree = this->graph.getInDegree(vertex);
        std::cout << "In Degree of Given Vertex: " << degree << std::endl;
    }
    catch (const std::exception &e)
    {
        std::cout << "Vertex " << vertex << " is not in the graph." << std::endl;
    }
}
```

```
void MenuController::displayOutDegreeOfGivenVertex() const
{
    Vertex vertex = this->ui.getUserVertex();

    try
    {
        unsigned int degree = this->graph.getOutDegree(vertex);
        std::cout << "Out Degree of Given Vertex: " << degree << std::endl;
    }
    catch (const std::exception &e)
    {
        std::cout << "Vertex " << vertex << " is not in the graph." << std::endl;
    }
}
```

```
void MenuController::addGivenVertexToGraph()
{
    Vertex vertex = this->ui.getUserVertex();

    if (this->graph.addVertex(vertex))
    {
        std::cout << "Successfully added Vertex" << std::endl;
    }
    else
    {
        std::cout << "FAILED to add Vertex" << std::endl;
    }
}
```

```
void MenuController::removeGivenVertexFromGraph()
{
    Vertex vertex = this->ui.getUserVertex();

    if (this->graph.removeVertex(vertex))
    {
        std::cout << "Successfully removed Vertex" << std::endl;
    }
    else
    {
        std::cout << "FAILED to remove Vertex" << std::endl;
    }
}
```

```
void MenuController::addGivenEdgeToGraph()
{
    Edge edge = this->ui.getUserEdge();
    EdgeCost cost = this->ui.getUserEdgeCost();

    if (this->ui.graph.addEdge(edge, cost))
    {
        std::cout << "Successfully added Edge" << std::endl;
    }
    else
    {
        std::cout << "FAILED to add Edge" << std::endl;
    }
}
```

```
void MenuController::removeGivenEdgeFromGraph()
{
    Edge edge = this->ui.getUserEdge();

    if (this->ui.graph.removeEdge(edge))
    {
        std::cout << "Successfully removed Edge" << std::endl;
    }
    else
    {
        std::cout << "FAILED to remove Edge" << std::endl;
    }
}
```

```
void MenuController::generateRandomGraphOfGivenNbVerticesAndNbEdges()
{
    unsigned int nbVertices = this->ui.getUserUnsignedInt("Number of Vertices: ");
    unsigned int nbEdges = this->ui.getUserUnsignedInt("Number of Edges: ");

    std::cout << "Generating a Random Graph....bz....z....." << std::endl;
    try
    {
        this->ui.graph = Helpers::generateRandomGraph(nbVertices, nbEdges);
    }
    catch (const std::exception &e)
    {
        std::cout << "Too many edges man." << std::endl;
    }
}
```

```
void MenuController::readGraphDataFromGivenFile()
{
    std::string filename = this->ui.getUserFilename();

    bool res = Helpers::readGraph(this->graph, filename);

    if (res)
    {
        std::cout << "Reading from " << filename << " was successful." << std::endl;
    }
    else
    {
        std::cout << "Reading from " << filename << " FAILED." << std::endl;
    }
}
```

```
void MenuController::writeGraphDataToGivenFile()
{
    std::string filename = this->ui.getUserFilename();

    bool res = Helpers::writeGraph(this->graph, filename);

    if (res)
    {
        std::cout << "Writing to " << filename << " was successful." << std::endl;
    }
    else
    {
        std::cout << "Writing to " << filename << " FAILED." << std::endl;
    }
}
```

Helpers

Class **Helpers** has no data members. It is used for storing static helper methods for the **ADTDirectedGraph** class.

Class **Helpers** has the following **methods**:

```
bool Helpers::readGraph(ADTDirectedGraph &graph, const std::string &filename)
{
    std::ifstream inputFile(filename);

    if (!inputFile.is_open())
    {
        return false;
    }

    unsigned int nbVertices, nbEdges;
    inputFile >> nbVertices >> nbEdges;

    Vertex first, second;
    EdgeCost cost;

    for (unsigned int i = 0; i < nbEdges; i++)
    {
        inputFile >> first >> second >> cost;

        graph.addVertex(first);
        graph.addVertex(second);

        graph.addEdge({first, second}, cost);
    }

    return true;
}
```

```
bool Helpers::writeGraph(const ADTDirectedGraph &graph, const std::string &filename)
{
    std::ofstream outputFile(filename);

    if (!outputFile.is_open())
    {
        return false;
    }

    outputFile << graph.nbVertices() << ' ' << graph.nbEdges() << std::endl;

    EdgeCosts::const_iterator startIt = graph.parseEdgesBegin();
    EdgeCosts::const_iterator endIt = graph.parseEdgesEnd();

    while (startIt != endIt)
    {

        Edge currEdge = startIt->first;
        EdgeCost currCost = startIt->second;

        outputFile << currEdge.first << ' ' << currEdge.second << ' ' << currCost << std::endl

        ++startIt;
    }

    return true;
}
```

```
ADTDirectedGraph Helpers::generateRandomGraph(unsigned int nbVertices, unsigned int nbEdges)
{

    // so if we have nbEdges > nbVertices * (nbVertices - 1) we have a problemo
    if (nbEdges > nbVertices * (nbVertices - 1))
    {
        throw std::exception();
    }

    ADTDirectedGraph graph = ADTDirectedGraph{};

    // way of generating numbers from 0 to the limit of unsigned int
    // rd is a seed gen, gen is a mersenne twister engine
    std::random_device rd;
    std::mt19937 gen(rd());
    std::uniform_int_distribution<unsigned int> randomUnsignedInt(0, UINT_MAX);
    std::uniform_int_distribution<unsigned int> randomVerticesIndex(0, nbVertices - 1);

    Vertices generatedVertices = Vertices{};

    // we generate nbVertices number of vertices
    for (unsigned int i = 0; i < nbVertices; ++i)
    {
        unsigned int vertex = Vertex{randomUnsignedInt(gen)};
        graph.addVertex(vertex);
        generatedVertices.push_back(vertex);
    }

    // we handle generating random edges
    unsigned int generatedEdges = 0;
    while (generatedEdges < nbEdges)
    {

        Vertex firstVertex = generatedVertices.at(randomVerticesIndex(gen));
        Vertex secondVertex = generatedVertices.at(randomVerticesIndex(gen));

        // if we got unlucky and we already have an edge we just skip
        if (graph.isEdge(firstVertex, secondVertex))
        {
            continue;
        }

        // we create an edge with unsigned int cost as well
        unsigned int randomCost = EdgeCost{randomUnsignedInt(gen)};
        Edge createdEdge = Edge{firstVertex, secondVertex};
        graph.addEdge(createdEdge, randomCost);
```

```
            generatedEdges++;
        }

        return graph;
    }
```